



01297 INTELLIGENT ROBOTICS AND AUTOMATION

[SPRING 24-25]

SECTION: A

SUBMISSION DATE: 03-06-25

PROJECT Report ON

Group 7

SMART GROCERY SHOPPING TROLLEY WITH PRODUCT SCANNING, WEIGHT VERIFICATION, AND HUMAN-FOLLOWING
CAPABILITY

SUBMITTED BY:

NAME	ID
MST. MAKSUDA AKTER SRABONI	22-47578-2
KAUSHIK RAY	22-46616-1
MD. ROBIUL ISLAM ROMAN	22-46490-1

Project Objectives:

Primary Goal: A smart trolley that detects the specific product automatically by measuring the weight and verifies by scanning the bar code of the product added to the cart, and displays the product's total price.

Secondary Goal: Follow a human with obstacle detection.

Problems That Are Solved:

- Shoppers often lose track of their total expenditure during shopping and only realize they have exceeded their budget at checkout. This situation can cause embarrassment or the need to return products. This issue is solved by the smart trolley, which continuously displays the total price and weight of selected items.
- Long waiting times at checkout counters caused by manual scanning and payment processes are resolved, as the trolley automates these functions.
- Human errors during the scanning and billing of products, which often lead to inaccurate totals and customer dissatisfaction, are effectively reduced through automated price calculation.
- Accessibility challenges faced by elderly or physically impaired shoppers who struggle with moving heavy trolleys and navigating through crowded aisles are addressed. The smart trolley is capable of following the user, thereby reducing physical effort.

Benefits That Are Provided:

- Real-time display of the total cost of items enables shoppers to manage their budget more effectively and avoid overspending.
- Checkout efficiency is improved, as automated price and weight display reduces time spent in queues.
- Billing accuracy is enhanced through automated product scanning, significantly lowering the possibility of human error.
- A more inclusive shopping experience is provided by reducing the physical burden on elderly and disabled users through the trolley's human-following capability.

Hardware Design and Specification:

Hardware used:

1. Raspberry Pi 3 Model B (4GB)
2. Load Cell + HX711 Amplifier
3. Raspberry Pi Camera Module V2 (8MP)
4. Ultrasonic Sensors (HC-SR04)
5. LCD Display (8x2)
6. DC Motors x2
7. L298N Motor Driver
8. Breadboard
9. Buck Converter
10. Wire



Figure 1: Raspberry pi

The Raspberry Pi is the central controller and the "brain" of our trolley system. It coordinates all hardware components, including sensors, motors, and the camera. It processes input from the ultrasonic sensor and Pi camera, runs the QR code detection algorithm, and sends appropriate signals to the motor driver and servo motor for movement and steering.



Figure 2: Ultrasonic Sensor

The ultrasonic sensor is used for obstacle detection. It continuously measures the distance to objects in front of the trolley. If it detects an object within 20 cm, it signals the Raspberry Pi to stop the trolley to avoid collisions, ensuring safe operation in dynamic environments.



Figure 3: Buck Converter

The buck converter is responsible for power regulation. It steps down the 12 V DC from the main battery to 5 V, which is required for powering the servo motor and other 5 V components in the system. This ensures energy efficiency and safe voltage levels for sensitive electronics.



Figure 4: Motor Controller

The motor controller (e.g., L298N) receives movement commands from the Raspberry Pi and drives the backward DC motors accordingly. It controls the speed and direction of the trolley's wheels, enabling it to move forward, backward, or stop based on the control logic.



Figure 5: Servo Motor

The servo motor adjusts the orientation of the Pi camera based on the position of the detected QR code. It enables the trolley to dynamically align the camera toward the human carrying the QR code, improving the accuracy and stability of the following behavior.



Figure 6: Load Sensor (HX711)

The HX711 load sensor module measures the weight of objects placed on the trolley. It uses a strain gauge (load cell) to detect changes in pressure, which the HX711 amplifies and converts to digital data. This data is sent to the Raspberry Pi to calculate the weight and generate a unique QR code displayed on the LCD, enhancing the trolley's functionality for weight-based tracking and logistics.



Figure 7: Pi Camera Module

The Pi camera captures live video and feeds it to the Raspberry Pi for image processing. It plays a key role in identifying and tracking the specific QR code assigned to the human user. Once the QR code is detected, the Raspberry Pi calculates its position and issues movement commands to follow it.



Figure 8: LCD Display

The LCD display is used to generate and display a QR code based on the weight placed on the trolley. A load sensor can be used to detect the weight, and the corresponding data is converted into a QR code that is shown on the LCD screen. This QR code can then be scanned and linked to delivery or tracking information, adding a layer of smart logistics to the system.

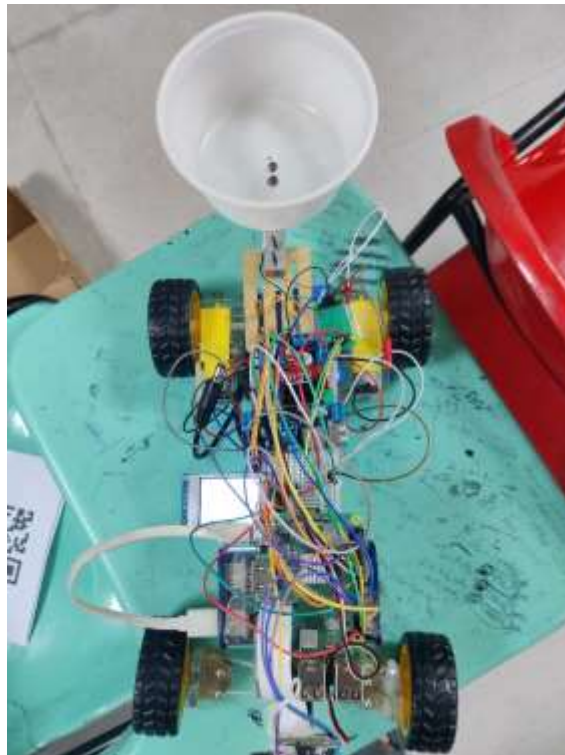


Figure 9: Full prototype (Smart trolley)

This is the full prototype of our project where all the sensors and motors are connected with the raspberry pi.

Software Design:

1. Human following code:

def main():

```
    GPIO.setmode(GPIO.BCM)
```

```
    robot = RobotController()
```

```
    camera = CameraSystem(robot)
```

```
    tracker = QRCodeTracker()
```

```
    ultrasonic = UltrasonicSensor(TRIG_PIN, ECHO_PIN)
```

```
    atexit.register(robot.cleanup)
```

```
    camera.start()
```

```
    logger.info("Starting QR tracking robot...")
```

```
    try:
```

```
        while True:
```

```
            frame = camera.get_frame()
```

```
            if frame is None:
```

```
                continue
```

```
            qr_data = tracker.detect_qr(frame)
```

```
            distance = ultrasonic.get_distance()
```

```
            if not camera.mode['manual'] and qr_data:
```

```
                tracker.last_seen_time = time.time()
```

```
                center_x, center_y = qr_data['center']
```

```
                qr_size = qr_data['size']
```

```
                frame_center = frame.shape[1] // 2
```

```
                # Steering logic based on QR position
```

```
                if center_x < frame_center - 50:
```

```
                    robot.steer(-30)
```

```
                elif center_x > frame_center + 50:
```

```
                    robot.steer(30)
```

```
                else:
```

```
                    robot.steer(0)
```

```
                # Movement control based on size and obstacle distance
```

```
                if qr_size < tracker.target_size - 20:
```

```

        if distance is not None and distance > 20:
            robot.move_forward(camera.mode['auto_speed'])
        else:
            robot.stop_motors()
            logger.info(f"Obstacle detected at {distance} cm")
    else:
        robot.stop_motors()
elif not camera.mode['manual'] and time.time() - tracker.last_seen_time > tracker.timeout:
    robot.stop_motors()
    robot.steer(0)
# Overlay information on video feed
if distance is not None:
    cv2.putText(frame, f"Distance: {distance}cm", (10, 60),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 0, 0), 2)
    cv2.putText(frame, f"Mode: {'MANUAL' if camera.mode['manual'] else 'AUTO'}",
                (10, 90), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 255), 2)
    cv2.putText(frame, f"Target Size: {tracker.target_size}px",
                (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
    cv2.line(frame, (frame.shape[1] // 2, 0),
              (frame.shape[1] // 2, frame.shape[0]), (0, 0, 255), 1)

    camera.update_stream(frame)

except KeyboardInterrupt:
    logger.info("Shutting down...")
finally:
    robot.cleanup()
    logger.info("Robot stopped")

```

The main() function represents the core operational loop for an autonomous robotic system designed to detect and follow QR codes using visual processing and obstacle avoidance. This function employs object-oriented principles to instantiate several interrelated components that form the robot's sensory and actuation system.

1. Initialization Phase

- The GPIO mode is configured using GPIO.setmode(GPIO.BCM), ensuring compatibility with Broadcom pin numbering.
- Instances of the key subsystems are created:

- RobotController() handles the motor and servo actuation.
 - CameraSystem(robot) manages real-time video capture and web interface.
 - QRCodeTracker() facilitates the detection of QR codes within frames.
 - UltrasonicSensor(TRIG_PIN, ECHO_PIN) measures distance to obstacles.
- The atexit module is employed to ensure the robot is safely shut down and GPIO pins are cleaned up on exit.

2. Main Control Loop

- Inside an infinite loop, the camera captures frames which are then processed for QR code recognition.
- If a QR code is detected and the system is in autonomous mode:
 - The robot computes the QR code's position and size.
 - Steering adjustments are made to align the QR code's horizontal center with the camera frame center.
 - The robot approaches the QR code if it is distant and no obstacles are detected using the ultrasonic sensor.
 - If the code is close or an obstacle is too near, the robot halts to avoid collisions.

3. Timeout Logic

- If a QR code has not been detected within a certain timeframe (tracker.timeout), the robot halts automatically. This prevents the robot from moving blindly.

4. User Interface Overlays

- The frame is annotated using cv2.putText() and cv2.line() to display useful information such as the robot's mode (manual or automatic), measured distance, and the QR code's desired size.
- This real-time annotation aids in monitoring system behavior visually.

2. Weight measuring code:

try:

while True:

Step 1: Read the average raw value from the load cell via HX711

raw_val = hx.read_average(10)

print(f"Raw: {raw_val}, Offset: {hx.OFFSET}")

Step 2: Compute the unclamped weight using the calibrated scale factor

weight_unclamped = (raw_val - hx.OFFSET) / hx.SCALE

print(f"Weight (unclamped): {weight_unclamped:.2f} g")

Step 3: Clamp the weight to ensure non-negative readings

weight = max(0, weight_unclamped)

print(f"Weight (clamped): {weight:.2f} g")


```
# Step 4: Generate a QR code that encodes the measured weight
```

```
qr_data = f"Weight: {weight:.2f} g"
```

```
qr_img = create_qr(qr_data).resize((128, 128)).convert("RGB")
```

```
# Step 5: Render the QR code on the TFT display
```

```
disp.image(qr_img)
```

```
# Step 6: Wait for a fixed interval before the next measurement
```

```
time.sleep(5)
```

```
except KeyboardInterrupt:
```

```
    print("Exiting...")
```

```
    GPIO.cleanup()
```

The control structure begins with a try-except block, facilitating a safe and graceful termination upon user interruption. Inside the while True loop, the system performs continuous monitoring and display of the measured weight.

1. **Sensor Reading:** The program uses a method `read_average()` to acquire multiple readings from the HX711 24-bit analog-to-digital converter connected to a load cell. By averaging multiple samples (10 in this case), transient fluctuations or electrical noise are minimized, improving measurement stability.
2. **Offset Compensation and Calibration:** The raw output is adjusted by subtracting a previously determined offset (`hx.OFFSET`). This offset represents the baseline value of the sensor when no load is applied. The adjusted value is then divided by a scaling factor (`hx.SCALE`), which is derived from prior calibration using a known reference mass. This calculation yields the actual mass in grams.
3. **Clamping Logic:** The result is constrained to non-negative values using Python's `max()` function. This step ensures that underflows or erroneous negative readings do not propagate into the displayed output.
4. **QR Code Generation:** A textual representation of the weight is encoded as a QR code using the `qrcode` library. The image is then resized and color-converted to match the display requirements of the ST7735 screen.
5. **Visual Output:** The generated QR code is rendered to the TFT display via the `disp.image()` method. This enables users to scan the weight data using a mobile device or embedded scanner, supporting traceability or inventory management.
6. **Temporal Delay:** The loop incorporates a five-second pause (`time.sleep(5)`), moderating the update rate to prevent excessive refresh cycles and reduce power consumption in embedded deployments.

3. OR code detection:

```
app = Flask(__name__)
```

```
# Initialize IP camera stream
```

```
ip_camera_url = 'http://192.168.159.126:8080/video'
```

```
cap = cv2.VideoCapture(ip_camera_url)
```

```
detector = cv2.QRCodeDetector()
```

```

# Global memory storage
scan_log = []
last_scanned_data = ""
last_scan_time = 0

# Background thread function for scanning QR codes
def scan_qr():
    global last_scanned_data, last_scan_time, scan_log
    while True:
        ret, img = cap.read()
        if not ret:
            continue

        data, bbox, _ = detector.detectAndDecode(img)
        if bbox is not None and data:
            now = datetime.now()
            timestamp = now.strftime("%d-%b-%Y %H:%M:%S")

            if data != last_scanned_data or (time.time() - last_scan_time) > 5:
                print("Scanned:", data, timestamp)
                scan_log.append((data, timestamp))
                with open('Database.csv', mode='a', newline='') as csvfile:
                    csv_writer = csv.writer(csvfile, delimiter=',', quotechar='"', quoting=csv.QUOTE_ALL)
                    csv_writer.writerow([data, timestamp])
                last_scanned_data = data
                last_scan_time = time.time()

# Define web interface route
@app.route('/')
def index():
    return render_template('index.html', data=scan_log[::-1])

# Entry point
if __name__ == '__main__':
    thread = threading.Thread(target=scan_qr, daemon=True)

```

```
thread.start()

app.run(host='0.0.0.0', port=5000, debug=False)
```

This program establishes a real-time QR code scanning and logging system integrated with a lightweight web interface for data visualization. The system architecture combines computer vision, concurrency, and web technologies to allow continuous monitoring of QR codes through an IP camera stream.

1. Camera Stream Initialization

The application begins by initiating a video stream from a user-defined IP camera address. This stream is acquired using a video capture interface that reads individual frames, enabling frame-by-frame processing. A QR code detector is instantiated to handle recognition and decoding tasks within each frame.

2. Global State Management

Several global variables are defined to maintain system state across different threads. `scan_log` acts as an in-memory list to store scanned QR code entries along with their respective timestamps. Additionally, `last_scanned_data` and `last_scan_time` variables are used to prevent repeated logging of the same code within a short time window, effectively implementing a debouncing mechanism.

3. Background Scanning Thread

A dedicated function, `scan_qr`, continuously retrieves frames from the video feed. If the frame capture is successful, it attempts to detect and decode a QR code from the current image. If a code is found and is either new or sufficiently delayed since the last scan, it is processed further. The result is:

- Printed to the console for real-time monitoring.
- Appended to an in-memory log for dynamic web display.
- Persistently stored in a CSV file named `Database.csv` for future analysis or archival purposes.

This thread runs independently in the background, allowing the web server to remain responsive.

4. Application Entry Point

The program's execution begins by checking if it is the main module. Upon confirmation, it launches the background QR scanning function in a separate thread using Python's threading module. The web server is then started, listening on all available network interfaces at port 5000.

Intelligence in the system:

- Autonomous QR Code Detection.
- Following specific human with the QR code.

Performance Results and Discussion:



Figure 10: QR code "A"



Figure 11: QR code

From these pictures it can be said that the trolley can follow the specific human. Here QR code "A" is detected because it is designed specifically for this trolley. On the other hand another QR code is not detected as it is not for this trolley.



Figure 12: Measuring the distance from Ultrasonic Sensor

When an object is in front of Ultrasonic sensor than the sensor value (distance) is getting matched with the measured value which is 19.6cm. That means the sensor is measuring the distance correctly.

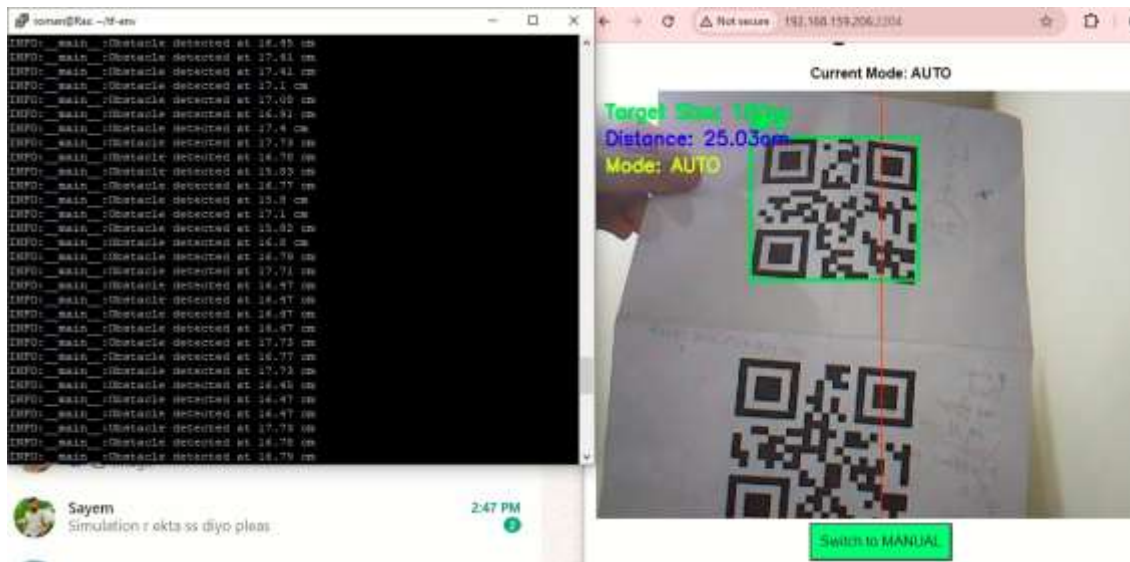


Figure 13: Obstacle detection with Ultrasonic sensor

The Ultrasonic sensor can detect the obstacle also.



Figure 14: Measuring weight



Figure 15: Measuring weight

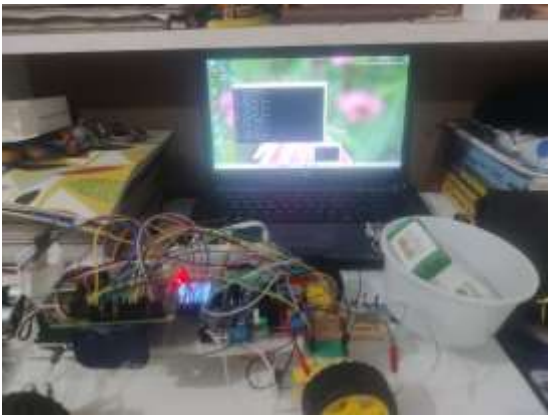


Figure 16: Measuring weight

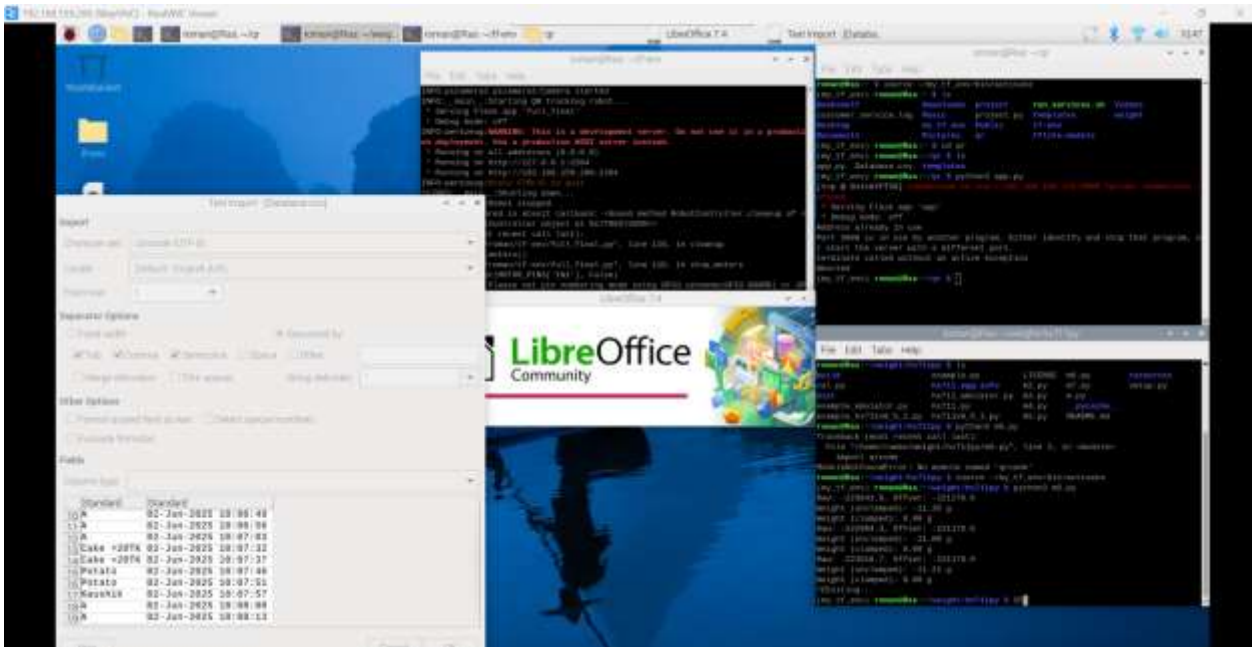


Figure 17

Scanned QR Code Data

Data	Timestamp
Posite	02-Jun-2025 18:07:51
Posite	02-Jun-2025 18:07:46
Cake +207K	02-Jun-2025 18:07:37
Cake +207K	02-Jun-2025 18:07:32
A	02-Jun-2025 18:07:25
A	02-Jun-2025 18:00:58
A	02-Jun-2025 18:00:48

Figure 18: QR Code Data



When measuring the weights of various items than a QR code is generated and when the QR code is scanned the weight can be seen in the display and different items have different weight. For example the weight of watch is 103gm, Soap is 263gm and Battery is 478.81gm and this weights can be seen in display.

Raw data table:

Raw	Offset	Net Δ	Approx Weight (g)
-188374	-221179	32805	228.13 g
-188358	-221179	32720.9	199.99 g
-188254	-221179	32924.5	200.62 g
-188266	-221179	32912.7	200.55 g
-188287	-221179	32891.2	200.42 g
-188343	-221179	32836	200.09 g

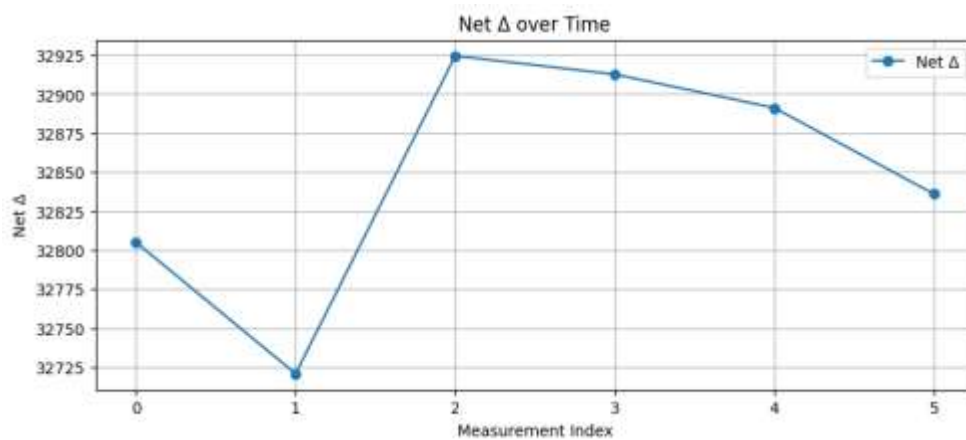


Figure 20: Net digital output over time

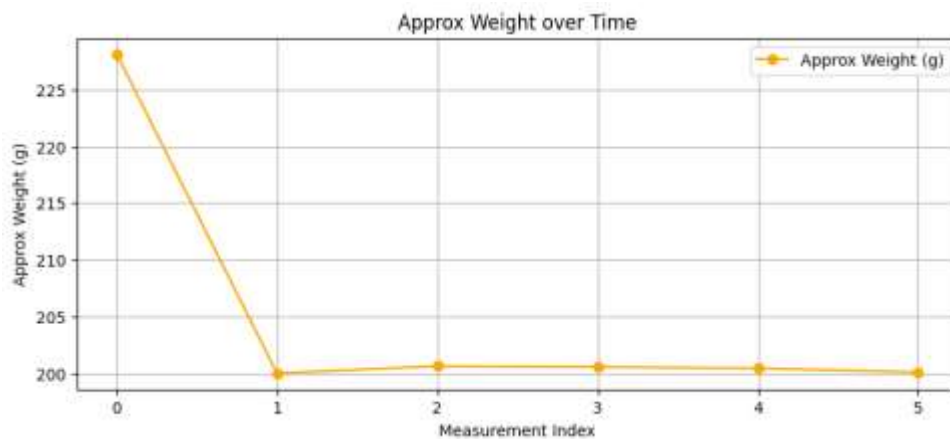


Figure 21: Approx weight over time

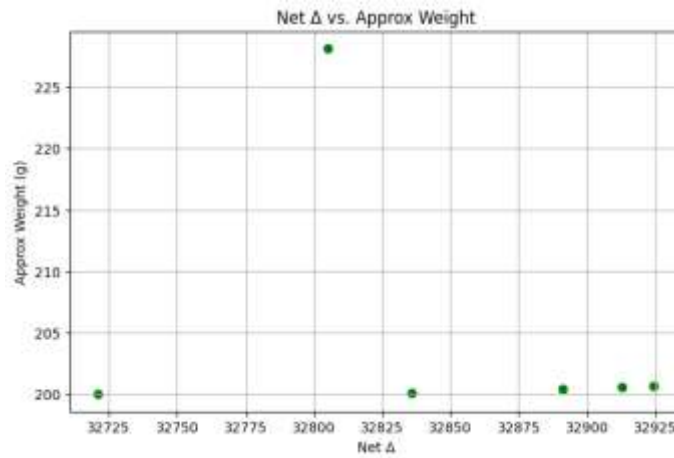


Figure 22: Net digital output vs. approx weight

In load cell-based measurement systems, raw data refers to the direct output from the ADC (such as HX711), which represents the signal corresponding to the applied weight. However, even without any load, the sensor may produce a non-zero reading due to mechanical or electrical bias—this is known as the offset data. To obtain accurate weight measurements, the offset is subtracted from the raw data, and the result is multiplied by a calibration factor. This process ensures the final output reflects only the actual load applied to the sensor.

Conclusion:

The primary objective of this project was to design and implement a smart trolley system capable of accurately measuring weight and follow specific human. This core functionality was successfully achieved. Throughout the development process, challenges were encountered in both the hardware and software components. These included issues related to sensor integration and programming logic, which were addressed through iterative testing and troubleshooting. A notable limitation was the failure of one of the camera modules, which prevented automatic QR code scanning for weight verification. As an interim solution, QR codes were scanned using a mobile phone. This aspect of the system can be improved in future versions by replacing the damaged camera module to fully restore autonomous operation. Overall, the project demonstrates a functional prototype that meets its primary objective and lays the groundwork for further enhancements.

References:

- [1] Sudipta Ranjan Subudhi, Ponnalagu R. N., "An Intelligent Shopping Cart with Automatic Product Detection and Secure Payment System," IEEE INDICON, 2019.
- [2] Shivika Srivastava et al., "IoT-based Human-Guided Smart Shopping Cart System for Shopping," Saudi Journal of Engineering and Technology, 2020.
- [3] K. G. Sawarkar, Sanchit Pandey, "Smart Shopping Cart for Physically Challenged," IJERT, 2022.

Appendix:

Human following code

```
import cv2
import numpy as np
import time
import threading
import atexit
import logging
from flask import Flask, Response, render_template_string
import RPi.GPIO as GPIO
from picamera2 import Picamera2
import pyzbar.pyzbar as pyzbar

# Configure logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

# GPIO Setup
SERVO_PIN = 17
MOTOR_PINS = {
    'IN1': 27, 'IN2': 22, 'IN3': 23, 'IN4': 24,
    'ENA': 25, 'ENB': 12
}
TRIG_PIN = 5
ECHO_PIN = 6

class UltrasonicSensor:
    def __init__(self, trig_pin, echo_pin):
        self.trig = trig_pin
        self.echo = echo_pin
        GPIO.setup(self.trig, GPIO.OUT)
        GPIO.setup(self.echo, GPIO.IN)
        GPIO.output(self.trig, False)
        time.sleep(0.1)

    def get_distance(self):
        GPIO.output(self.trig, True)
        time.sleep(0.00001)
        GPIO.output(self.trig, False)

        timeout = time.time() + 0.04
        start = time.time()
        while GPIO.input(self.echo) == 0:
            start = time.time()
            if start > timeout:
                return None

        end = time.time()
        while GPIO.input(self.echo) == 1:
            end = time.time()
            if end > timeout:
                return None
```

```
duration = end - start
distance = duration * 17150
return round(distance, 2)
```

```
class RobotController:
```

```
    def __init__(self):
        self._setup_gpio()
        self._setup_servo()
        self._setup_motors()
        self.last_duty = None
```

```
    def _setup_gpio(self):
        GPIO.setmode(GPIO.BCM)
        GPIO.setwarnings(False)
```

```
    def _setup_servo(self):
        GPIO.setup(SERVO_PIN, GPIO.OUT)
        self.servo = GPIO.PWM(SERVO_PIN, 50)
        self.servo.start(0)
        time.sleep(0.5)
```

```
    def _setup_motors(self):
        for pin in MOTOR_PINS.values():
            GPIO.setup(pin, GPIO.OUT)
        self.left_pwm = GPIO.PWM(MOTOR_PINS['ENA'], 1000)
        self.right_pwm = GPIO.PWM(MOTOR_PINS['ENB'], 1000)
        self.left_pwm.start(0)
        self.right_pwm.start(0)
```

```
    def steer(self, angle):
        angle = max(-90, min(90, angle))
        duty = 2.5 + (angle + 90) * (10.0 / 180.0)
        duty = round(duty, 1)
```

```
        if duty != self.last_duty:
            self.servo.ChangeDutyCycle(duty)
            time.sleep(0.15)
            self.servo.ChangeDutyCycle(0)
            self.last_duty = duty
```

```
    def move_forward(self, speed=50):
        speed = max(30, min(70, speed))
        self.left_pwm.ChangeDutyCycle(speed)
        self.right_pwm.ChangeDutyCycle(speed)
        GPIO.output(MOTOR_PINS['IN1'], True)
        GPIO.output(MOTOR_PINS['IN2'], False)
        GPIO.output(MOTOR_PINS['IN3'], True)
        GPIO.output(MOTOR_PINS['IN4'], False)
```

```
    def stop_motors(self):
        GPIO.output(MOTOR_PINS['IN1'], False)
        GPIO.output(MOTOR_PINS['IN2'], False)
        GPIO.output(MOTOR_PINS['IN3'], False)
        GPIO.output(MOTOR_PINS['IN4'], False)
        self.left_pwm.ChangeDutyCycle(0)
        self.right_pwm.ChangeDutyCycle(0)
```

```

def cleanup(self):
    self.stop_motors()
    self.steer(0)
    self.servo.stop()
    self.left_pwm.stop()
    self.right_pwm.stop()
    GPIO.cleanup()

```

```

class QRCodeTracker:

```

```

    def __init__(self):
        self.target_size = 150
        self.last_seen_time = 0
        self.timeout = 3.0

```

```

    def detect_qr(self, frame):
        decoded = pyzbar.decode(frame)
        for code in decoded:
            data = code.data.decode('utf-8')
            if data.strip().upper() == 'A':
                x, y, w, h = code.rect
                cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
                cv2.putText(frame, "QR A", (x, y-10),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)
                return {
                    'center': (x + w//2, y + h//2),
                    'size': max(w, h),
                    'data': data
                }
        return None

```

```

class CameraSystem:

```

```

    def __init__(self, robot):
        self.camera = Picamera2()
        self.stream = Flask(__name__)
        self.frame = None
        self.lock = threading.Lock()
        self.robot = robot
        self.mode = {
            'manual': False,
            'speed': 50,
            'auto_speed': 40
        }

```

```

        config = self.camera.create_preview_configuration(main={"size": (640, 480)})
        self.camera.configure(config)

```

```

        self.stream.add_url_rule('/', 'index', self._index)
        self.stream.add_url_rule('/video_feed', 'video_feed', self._video_feed)
        self.stream.add_url_rule('/control/<action>', 'control', self._control)

```

```

    def start(self):
        self.camera.start()
        time.sleep(2)
        threading.Thread(target=self.stream.run,
                        kwargs={'host': '0.0.0.0', 'port': 2204},

```

```

        daemon=True).start()

def get_frame(self):
    frame = self.camera.capture_array()
    return cv2.cvtColor(frame, cv2.COLOR_RGB2BGR)

def update_stream(self, frame):
    with self.lock:
        self.frame = frame.copy()

def _index(self):
    manual_display = 'block' if self.mode['manual'] else 'none'
    auto_display = 'block' if not self.mode['manual'] else 'none'
    manual_js_flag = 'true' if self.mode['manual'] else 'false'
    return render_template_string(f"""
    <html>
    <head>
        <title>QR Tracking Robot</title>
        <style>
            body {{ font-family: Arial, sans-serif; text-align: center; }}
            button {{ margin: 5px; padding: 10px; font-size: 16px; min-width: 100px; }}
            #manualControls, #autoControls {{ margin: 20px auto; max-width: 500px; }}
            .slider-container {{ margin: 15px 0; }}
            .status {{ margin: 10px; font-weight: bold; }}
        </style>
    </head>
    <body>
        <h1>QR Tracking Robot Controller</h1>
        <div class="status">Current Mode: {'MANUAL' if self.mode['manual'] else 'AUTO'}</div>
        <br>

        <button onclick="sendCommand('toggle')" style="background: {'#ff5555' if self.mode['manual'] else
'#55ff55'};">
            Switch to {'AUTO' if self.mode['manual'] else 'MANUAL'}
        </button>

        <div id="manualControls">
            <div class="slider-container">
                <label for="speedRange">Manual Speed: <span
id="speedVal">{self.mode['speed']}</span></label><br>
                <input type="range" id="speedRange" min="30" max="70" value="{self.mode['speed']}"
step="1">
            </div>
            <button onclick="sendCommand('forward')">Forward (W)</button><br>
            <button onclick="sendCommand('left')">Left (A)</button>
            <button onclick="sendCommand('stop')">Stop (S)</button>
            <button onclick="sendCommand('right')">Right (D)</button>
            <button onclick="sendCommand('center')">Center</button>
        </div>

        <div id="autoControls">
            <div class="slider-container">
                <label for="autoSpeedRange">Auto Speed: <span
id="autoSpeedVal">{self.mode['auto_speed']}</span></label><br>
                <input type="range" id="autoSpeedRange" min="30" max="70"
value="{self.mode['auto_speed']}" step="1">

```

```

    </div>
    <button onclick="sendCommand('stop')" style="background: #ff5555;">Emergency Stop
(S)</button>
</div>

```

```

<script>
    let speed = {self.model['speed']};
    let autoSpeed = {self.model['auto_speed']};
    const speedRange = document.getElementById('speedRange');
    const speedVal = document.getElementById('speedVal');
    const autoSpeedRange = document.getElementById('autoSpeedRange');
    const autoSpeedVal = document.getElementById('autoSpeedVal');

```

```

    speedRange.addEventListener('input', () => {{
        speed = speedRange.value;
        speedVal.textContent = speed;
        fetch(`/control/speed/${{speed}}`);
    }});

```

```

    autoSpeedRange.addEventListener('input', () => {{
        autoSpeed = autoSpeedRange.value;
        autoSpeedVal.textContent = autoSpeed;
        fetch(`/control/auto_speed/${{autoSpeed}}`);
    }});

```

```

    function sendCommand(action) {{
        if (action === 'toggle') {{
            fetch('/control/toggle')
                .then(response => {{
                    if (response.ok) {{
                        window.location.reload();
                    }}
                }});
        }} else {{
            fetch('/control/' + action);
        }}
    }}

```

```

    document.addEventListener('keydown', function(event) {{
        if (!{manual_js_flag}) return;
        const key = event.key.toLowerCase();
        if (['w','a','s','d'].includes(key)) {{
            event.preventDefault();
            const cmd = {{'w':'forward','a':'left','s':'stop','d':'right'}}[key];
            sendCommand(cmd);
        }}
    }});

```

```

</script>
</body>
</html>

```

```

""")

```

```

def _video_feed(self):
    def generate():
        while True:
            with self.lock:

```

```

        if self.frame is None:
            continue
        ret, jpeg = cv2.imencode('.jpg', self.frame)
        if not ret:
            continue
        yield (b'--frame\r\n'
              b'Content-Type: image/jpeg\r\n\r\n' +
              jpeg.tobytes() + b'\r\n')
    return Response(generate(),
                    mimetype='multipart/x-mixed-replace; boundary=frame')

```

```

def _control(self, action):
    if action == 'toggle':
        self.mode['manual'] = not self.mode['manual']
        self.robot.stop_motors()
        self.robot.steer(0)
        logger.info(f'Mode toggled to {'Manual' if self.mode['manual'] else 'Auto'}')
        return Response(status=204)

```

```

    elif action.startswith('speed/'):
        try:
            speed_val = int(action.split('/')[1])
            self.mode['speed'] = max(30, min(70, speed_val))
            logger.info(f'Manual speed set to {self.mode['speed']}')
        except Exception as e:
            logger.error(f'Failed to set speed: {e}')

```

```

    elif action.startswith('auto_speed/'):
        try:
            speed_val = int(action.split('/')[1])
            self.mode['auto_speed'] = max(30, min(70, speed_val))
            logger.info(f'Auto speed set to {self.mode['auto_speed']}')
        except Exception as e:
            logger.error(f'Failed to set auto speed: {e}')

```

```

    elif self.mode['manual']:
        if action == 'forward':
            self.robot.move_forward(self.mode['speed'])
        elif action == 'stop':
            self.robot.stop_motors()
        elif action == 'left':
            self.robot.steer(-30)
        elif action == 'right':
            self.robot.steer(30)
        elif action == 'center':
            self.robot.steer(0)

```

```

    elif action == 'stop':
        self.robot.stop_motors()

```

```

    return ("", 204)

```

```

def main():
    GPIO.setmode(GPIO.BCM)
    robot = RobotController()
    camera = CameraSystem(robot)

```

```

tracker = QRCodeTracker()
ultrasonic = UltrasonicSensor(TRIG_PIN, ECHO_PIN)

atexit.register(robot.cleanup)
camera.start()

logger.info("Starting QR tracking robot...")

try:
    while True:
        frame = camera.get_frame()
        if frame is None:
            continue

        qr_data = tracker.detect_qr(frame)
        distance = ultrasonic.get_distance()

        if not camera.mode['manual'] and qr_data:
            tracker.last_seen_time = time.time()
            center_x, center_y = qr_data['center']
            qr_size = qr_data['size']
            frame_center = frame.shape[1] // 2

            if center_x < frame_center - 50:
                robot.steer(-30)
            elif center_x > frame_center + 50:
                robot.steer(30)
            else:
                robot.steer(0)

            if qr_size < tracker.target_size - 20:
                if distance is not None and distance > 20:
                    robot.move_forward(camera.mode['auto_speed'])
                else:
                    robot.stop_motors()
                    logger.info(f"Obstacle detected at {distance} cm")
            else:
                robot.stop_motors()
        elif not camera.mode['manual'] and time.time() - tracker.last_seen_time > tracker.timeout:
            robot.stop_motors()
            robot.steer(0)

        # Display info
        if distance is not None:
            cv2.putText(frame, f"Distance: {distance}cm", (10, 60),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 0, 0), 2)
        cv2.putText(frame, f"Mode: {'MANUAL' if camera.mode['manual'] else 'AUTO'}",
                    (10, 90), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 255), 2)
        cv2.putText(frame, f"Target Size: {tracker.target_size}px",
                    (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
        cv2.line(frame, (frame.shape[1]//2, 0),
                 (frame.shape[1]//2, frame.shape[0]), (0, 0, 255), 1)

        camera.update_stream(frame)

except KeyboardInterrupt:

```

```

    logger.info("Shutting down...")
finally:
    robot.cleanup()
    logger.info("Robot stopped")

if __name__ == "__main__":
    main()

```

Weight measurement code

```

import time
import qrcode
from PIL import Image
import board
import digitalio
import adafruit_rgb_display.st7735 as st7735
import RPi.GPIO as GPIO

# ===== HX711 CLASS =====
class HX711:
    def __init__(self, dout, pd_sck, gain=128):
        self.PD_SCK = pd_sck
        self.DOUT = dout

        GPIO.setmode(GPIO.BCM)
        GPIO.setup(self.PD_SCK, GPIO.OUT)
        GPIO.setup(self.DOUT, GPIO.IN)

        self.GAIN = 0
        self.OFFSET = 0
        self.SCALE = 1

        self.set_gain(gain)

    def is_ready(self):
        return GPIO.input(self.DOUT) == 0

    def set_gain(self, gain):
        if gain == 128:
            self.GAIN = 1
        elif gain == 64:
            self.GAIN = 3
        elif gain == 32:
            self.GAIN = 2
        GPIO.output(self.PD_SCK, False)
        self.read()

    def read(self):
        while not self.is_ready():
            pass
        data = [0] * 3
        for j in range(3):
            data[j] = 0
            for i in range(8):
                GPIO.output(self.PD_SCK, True)
                data[j] <<= 1
                GPIO.output(self.PD_SCK, False)

```



```

        if GPIO.input(self.DOUT):
            data[j] |= 1
    for _ in range(self.GAIN):
        GPIO.output(self.PD_SCK, True)
        GPIO.output(self.PD_SCK, False)
    value = ((data[0] << 16) | (data[1] << 8) | data[2])
    if value & 0x800000:
        value |= ~0xfffff
    return value

def read_average(self, times=3):
    values = [self.read() for _ in range(times)]
    return sum(values) / len(values)

def read_median(self, times=3):
    valueList = [self.read() for _ in range(times)]
    valueList.sort()
    midpoint = len(valueList) // 2
    if len(valueList) % 2 == 0:
        return sum(valueList[midpoint:midpoint+2]) / 2.0
    else:
        return valueList[midpoint]

def tare(self, times=15):
    self.OFFSET = self.read_average(times)

def get_value(self, times=3):
    return self.read_average(times) - self.OFFSET

def get_weight(self, times=3):
    return self.get_value(times) / self.SCALE

def set_offset(self, offset):
    self.OFFSET = offset

def set_scale(self, scale):
    self.SCALE = scale

def power_down(self):
    GPIO.output(self.PD_SCK, False)
    GPIO.output(self.PD_SCK, True)

def power_up(self):
    GPIO.output(self.PD_SCK, False)

# ===== MAIN SCRIPT =====
hx = HX711(20, 21)
hx.set_scale(-2.77) # Calibrated scale factor for 250g known weight
hx.tare()

# Initialize ST7735 display
cs_pin = digitalio.DigitalInOut(board.CE0)
dc_pin = digitalio.DigitalInOut(board.D16)
reset_pin = digitalio.DigitalInOut(board.D26)

disp = st7735.ST7735R(

```

```

board.SPI(),
cs=cs_pin,
dc=dc_pin,
rst=reset_pin,
width=128,
height=160,
rotation=270,
)
disp.fill(0)

def create_qr(data):
    qr = qrcode.QRCode(
        version=1,
        box_size=4,
        border=1
    )
    qr.add_data(data)
    qr.make(fit=True)
    img = qr.make_image(fill_color="black", back_color="white")
    return img

try:
    while True:
        # Read raw value
        raw_val = hx.read_average(10)
        print(f"Raw: {raw_val}, Offset: {hx.OFFSET}")

        # Calculate weight
        weight_unclamped = (raw_val - hx.OFFSET) / hx.SCALE
        print(f"Weight (unclamped): {weight_unclamped:.2f} g")

        weight = max(0, weight_unclamped)
        print(f"Weight (clamped): {weight:.2f} g")

        # Create QR code
        qr_data = f"Weight: {weight:.2f} g"
        qr_img = create_qr(qr_data).resize((128, 128)).convert("RGB")

        # Update display
        disp.image(qr_img)

        # Wait 5 seconds
        time.sleep(5)

except KeyboardInterrupt:
    print("Exiting...")
    GPIO.cleanup()

```

Qr Code

```

import cv2
import csv
import time
import threading
from flask import Flask, render_template
from datetime import datetime

```

```

app = Flask(__name__)

# IP Camera stream (replace this with your phone's URL)
ip_camera_url = 'http://192.168.159.126:8080/video'
cap = cv2.VideoCapture(ip_camera_url)
detector = cv2.QRCodeDetector()

# Global storage
scan_log = []
last_scanned_data = ""
last_scan_time = 0

# Background thread for scanning QR codes
def scan_qr():
    global last_scanned_data, last_scan_time, scan_log
    while True:
        ret, img = cap.read()
        if not ret:
            continue

        data, bbox, _ = detector.detectAndDecode(img)
        if bbox is not None and data:
            now = datetime.now()
            timestamp = now.strftime("%d-%b-%Y %H:%M:%S")

            if data != last_scanned_data or (time.time() - last_scan_time) > 5:
                print("Scanned:", data, timestamp)

                # Log to memory and file
                scan_log.append((data, timestamp))
                with open('Database.csv', mode='a', newline='') as csvfile:
                    csv_writer = csv.writer(csvfile, delimiter=',', quotechar='"', quoting=csv.QUOTE_ALL)
                    csv_writer.writerow([data, timestamp])

                last_scanned_data = data
                last_scan_time = time.time()

# Web route to display logs
@app.route('/')
def index():
    return render_template('index.html', data=scan_log[::-1]) # show latest first

# Start scanning thread before Flask app
if __name__ == '__main__':
    thread = threading.Thread(target=scan_qr, daemon=True)
    thread.start()
    app.run(host='0.0.0.0', port=5000, debug=False)

```